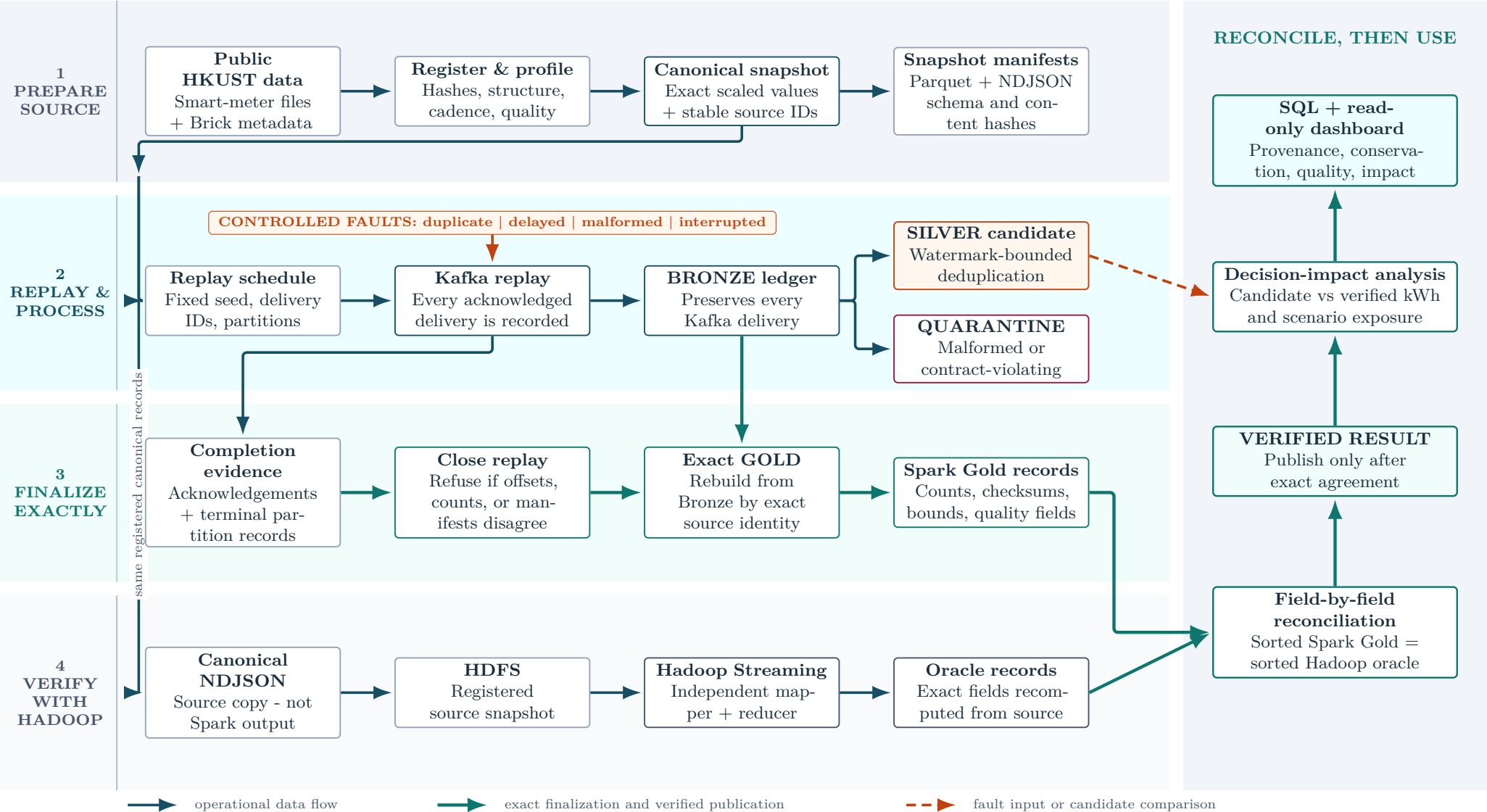


Research question: A distributed job has completed. How do we prove that every delivery was handled correctly and that the resulting decision is based on independently verified data?



Core idea: Completing a Spark job is not proof of data correctness. The pipeline accounts for every delivery, reconstructs exact results, checks them through an independently implemented Hadoop computation, and measures the decision impact of errors that an operational stream may miss.

What we are doing

1. **Fix the source.** Register public HKUST smart-meter data and Brick metadata; profile file hashes, workbook structure, timestamps, cadence, precision, missingness, and duplicate timestamps.
2. **Preserve exact identity and value.** Create deterministic Parquet and NDJSON snapshots. Each row keeps its source file, sheet, physical row, original text, and decimal value as a scaled integer.
3. **Replay realistic delivery faults.** Use fixed seeds to produce clean, duplicate, delayed, malformed, combined, and interrupted Kafka deliveries. Delivery acknowledgements and terminal partition records establish what was actually sent.
4. **Separate operational and final truth.** Spark Bronze preserves all deliveries. Silver gives a fast, watermark-bounded candidate; invalid data goes to Quarantine. After replay completion, Gold is rebuilt exactly from Bronze using source identity.
5. **Verify before use.** Hadoop independently recomputes the declared records from canonical NDJSON. Only reconciled results feed decision-impact reports, SQL queries, and the read-only dashboard.

Why this is novel

The contribution is the combined assurance method, not simply the use of Kafka, Spark, and Hadoop.

- **Job success is replaced by conservation evidence.** Every source event and acknowledged delivery has a deterministic identity and an auditable destination; missing terminals, count mismatches, or manifest disagreement prevent finalization.
- **Operational truth and exact truth are explicit.** A bounded Silver result is retained as the realistic candidate, while exact Gold is rebuilt only after the replay is demonstrably complete.
- **Verification is implementation-diverse.** Hadoop reads the registered source rather than Spark output and does not import Spark transformation code. Agreement is checked on sorted exact fields.
- **Faults have known ground truth.** Reproducible injection makes duplicate handling, late-data behaviour, quarantine, and restart equivalence measurable instead of anecdotal.
- **Correctness is connected to decisions.** Candidate-versus- verified differences are reported as affected meter-periods, absolute kWh discrepancy, and normalized low/base/high billing scenarios.

Future expansion

If coursework time permits

- Complete the evidence-based real-dataset audit, assigning timezone and cumulative-kWh semantics only where the source supports them.
- Run the accepted Docker fixture, then designated-machine experiments at 100K, 1M, and up to 5M/full scale. Measure throughput, recovery, watermarks, storage formats, and query performance.
- Finalize the generated report, read-only dashboard, and 5-7 minute demonstration using prepared results rather than live large workloads.

Proposed MTP direction

Risk-Adaptive Independent Verification for Decision-Critical IoT Data Pipelines

- Allocate second-engine verification to high-risk meter-time windows using data quality, reset, rollover, gap, metadata, and decision-impact signals.
- Optimize verification coverage against latency, compute, and storage budgets; compare full checking, fixed sampling, and adaptive selection.
- Extend to multi-engine oracles, fault localization, online correction, and stronger provenance attestations.
- Evaluate generalization across buildings, institutions, and additional public IoT datasets.

MTP research question: How can a pipeline spend verification effort where it most reduces expected decision loss while maintaining auditable correctness under a fixed resource budget?

Follow the numbered central path from source registration to publication. The left branch is the independent Hadoop check; the right branch preserves the operational Spark outputs and blocks publication when exact records disagree.

